ServiceSync - Notification Server

This page describes why a server was needed, how to run it on your own machine and all the technical details about how it was envisioned and implemented.

Spring Boot - Spring Boot

Initializer - https://start.spring.io/

Spring Annotations - Annotations :: Spring Framework

Java SDK 23 - Download the Latest Java LTS Free

Type	Key	Summary	Assignee	Priority	Sta
	FTR-116	Notification server starts on fixed port number		Medium	DO
	FTR-88	Implement a server for notifications		Medium	DO

2 items Synced just now

WARNING: This server is not open for modification outside of our team members, contact us for enchancement and improvement before starting.

Why did ServiceSync need a server?
Server tech stack
How to run using the source code
How to run the server using the .jar file

How it works (technical details)
- Interfaces
  - IReservationDataProvider
  - IMessagingProvider
- Configurations
  - FirestoreConfig
  - FirestoreMessagingConfig
  - SchedulingConfig
  - PortListener
- Data classes
  - Reservation
  - NotificationData
- Data access layer
  - FirestoreManager
  - FirebaseMessagingManager
- Business logic layer
  - ReservationService
- SpringApplication component scan

Diagrams
- Package diagram
- Class Diagram

Why did ServiceSync need a server?

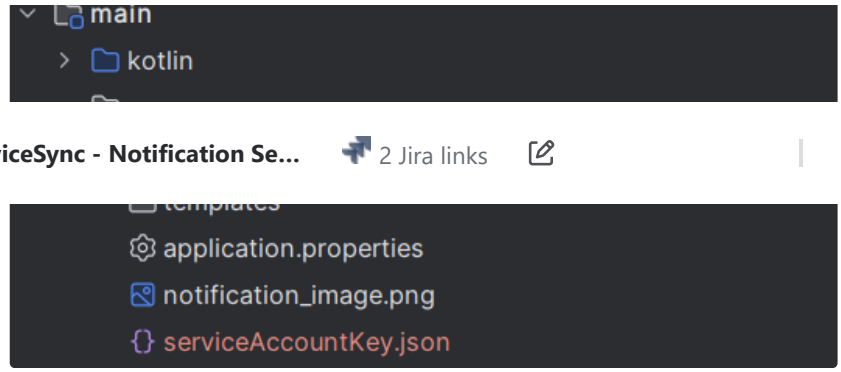
Because of the notification functionality.

Server tech stack

- Spring Boot with **Kotlin**
- Firestore
- Firebase Cloud Messaging

How to run using the source code

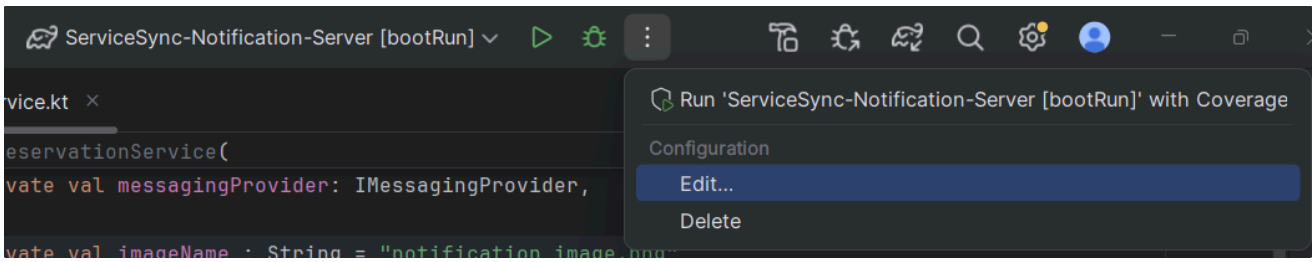
1. Clone the repository form the **main** branch (link)
2. After cloning the repository, add the provided serviceAccount.json in **main/resources** of the project

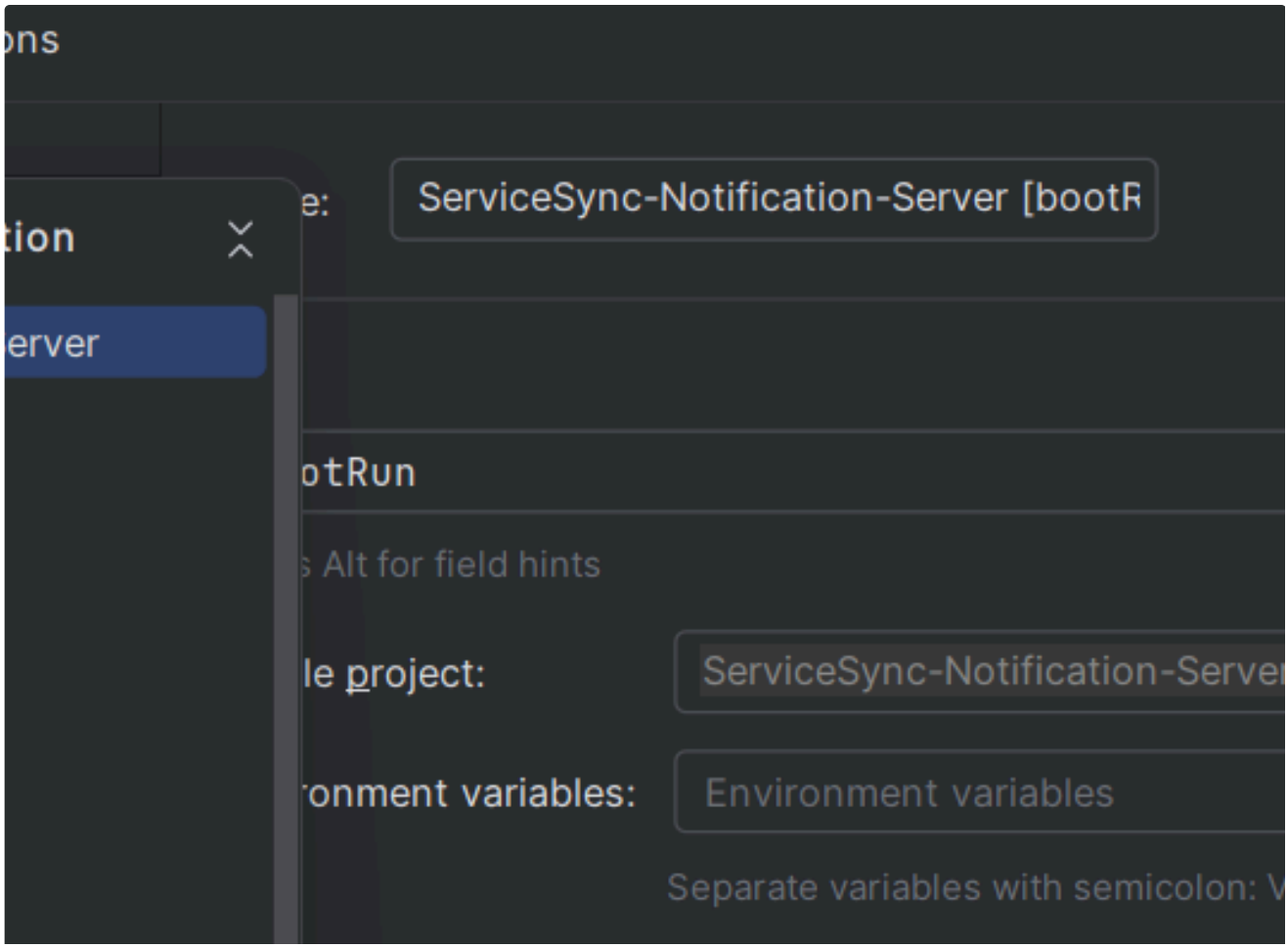


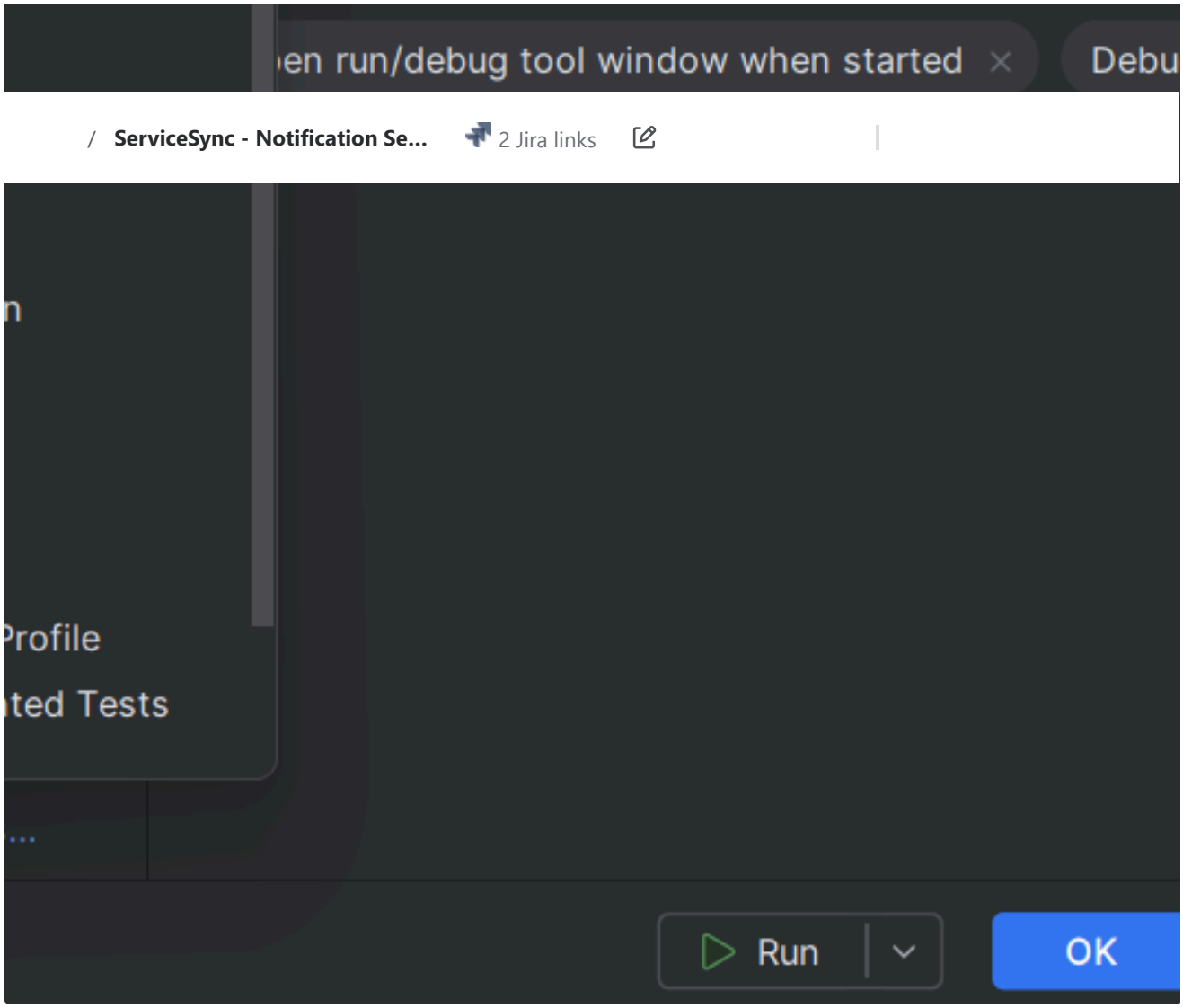
/ **ServiceSync - Notification Se...** 2 Jira links

serviceAccountKey.json
02 Jan 2025, 01:43 PM

3. Run the project - if you are using Android Studio, you can add configuration options for running






en run/debug tool window when started
Debu
/ ServiceSync - Notification Se...
2 Jira links
n
Profile
ted Tests
Run
OK

/ **ServiceSync - Notification Se...** 2 Jira links

You are all set! The server fill try to fetch the reservations every **1 hour**. The run window will display extra logging information.

How to run the server using the .jar file

Install Java SDK 23 - https://www.oracle.com/java/technologies/downloads/?er=221886#jdk23-windows

Place the .jar somewhere on your system and execute the following command:

```
java -jar <path>/ServiceSync-Notification-Server.jar
```

This will start the notification server on the first available port it finds.

How it works (technical details)

This section of the page is meant for everyone wanting to understand how this is currently implemented and how it works. Let us start with the data access layer that communicated with **Firestore** and **Firebase Messaging.**

As always, data access layer has 3 custom interfaces that enable the server to be more resistant to changes and future-proof.

Interfaces

IReservationDataProvider

```
interface IReservationDataProvider {
    fun getAllUnsentNotifications(
        minThreshold : Long,
        maxThreshold: Long,
        onSuccess: (List<Reservation>) -> Unit
    )

    fun markNotificationAsSent(successfullySentReservationNotifications: List<Reservat
}
```

The purpose of IReservationDataProvider is to enforce any class that implements it to be able to fetch reservations from some source and mark them later on as sent (their notification has been successfully sent).

Methods:

- **getAllUnsentNotifications** - like the name suggests, this method received a min and max threshold which are long (these are expected to be dates converted to miliseconds, min is the minimum time, while max is the maximum allowed time). As you will see later on, min threshold is the time when the query is executed and max is the time when the query is executed, but with added 24 hours. OnSuccess is a required method that will be called when the query executes successfully and returns a list of all reservations waiting for their notifications to be sent.
- **markNotificationAsSent** - this method receives a list of all reservations that had their notifications sent, so the class should mark them as sent

IFcmTokenProvider

```
interface IFcmTokenProvider {
    fun getFcmToken(username: String) : String
}
```

- **getFcmToken** - this method receives a username for which a fcm token is required. It must return the token in the string format

/ **ServiceSync - Notification Se...** 2 Jira links

```
interface IMessagingProvider {
    fun sendNotification(notificationData: List<NotificationData>,
    imageURL: String,
    onSuccessfullySentNotifications: (List<NotificationData>) -> Unit)
}
```

- **sendNotification -** this method receives a list of all notifications to be sent, path to the image that will be used to send a notification and a funciton that will be executed after this method finds all successfully sent notifications and returns them to the business logic layer

---

Configurations

FirestoreConfig

```
@Configuration
class FirestoreConfig {
    @Bean
    fun firestore() : Firestore {
        val serviceAccount: InputStream = this::class.java.getResourceAsStream("/serv
            ?: throw IllegalStateException("Service account key file not found in cla

        val options = FirebaseOptions.builder()
            .setCredentials(GoogleCredentials.fromStream(serviceAccount))
            .setProjectId("airtag-servicesync")
            .build()

        FirebaseApp.initializeApp(options)

        return FirestoreClient.getFirestore()
    }

}
```

This configuration class will be execute whenever another class requires a object of class Firestore. It will load the service account json, set the credentials and project id, load the firebase app and return a active Firestore object.

FirestoreMessagingConfig

```
@Configuration
class FirebaseMessagingConfig {
    @Bean
```

```
    fun firebaseMessaging() : FirebaseMessaging {
        return FirebaseMessaging.getInstance()
```

/ **ServiceSync - Notification Se...** 2 Jira links

Returns an instance to FirebaseMessaging

SchedulingConfig

```
@Configuration
@EnableScheduling
class SchedulingConfig {
}
```

This class enables Scheduling across the Spring app.

PortListener

```
@Component
class PortListener {
    @EventListener
    fun onApplicationStart(event: WebServerInitializedEvent) {
        val port = event.webServer.port
        println("ServiceSync Notification Server is running on port: ${port}")
    }
}
```

This component prints the port on which the notification server was started. Gets the port from application.properties:

```
server.port=0
```

Data classes

Reservation

```
data class Reservation(
    val companyId : String = "",
    val serviceName: String = "",
    val reservationDate: Long = 0,
    val notificationSent: Boolean = false,
    val userId : String = ""
)
```

This class is used to transform Firestore documents to a custom class (all field are identitical to document fields in Firestore). They have default parameters, because Firestore create a object and later on fills it with actual values. This class is used on the business logic layer and as a mean of communication between business and data layer.

NotificationData

```
data class NotificationData(
```

/ **ServiceSync - Notification Se...** 2 Jira links

```
    val body: String
)
```

This data class is to tell business logic layer which data the messaging provider needs to send a notification.

Data access layer

FirestoreManager

```
@Repository
class FirestoreManager(private val firestore: Firestore) : IReservationDataProvider,I
    private val reservationCollectionName = "reservations"
    private val reservationDateField = "reservationDate"
    private val userCollectionName = "users"
    private val fcmTokenField = "FCMToken"
    private val notificationSentField = "notificationSent"
    private val reservationDocumentMap = mutableMapOf<Reservation,QueryDocumentSnapsh

    override fun getAllUnsentNotifications(minThreshold : Long, maxThreshold: Long, c
        try {
            val unsentNotifications = firestore
                .collection(reservationCollectionName)
                .whereGreaterThan(reservationDateField,minThreshold)
                .whereLessThanOrEqualTo(reservationDateField,maxThreshold)
                .whereEqualTo(notificationSentField,false)
                .get()
                .get()
                .documents

            if (unsentNotifications.isNotEmpty()) {
                val reservations = mutableListOf<Reservation>()
                unsentNotifications.forEach { unsentNotification ->
                    val reservation = unsentNotification.toObject(Reservation::class.
                    reservations.add(reservation)
                    reservationDocumentMap[reservation] = unsentNotification
                }
                onSuccess(reservations)
            }
        }
        catch (e : Exception) {
            println("Error while fetching unsent notifications: ${e.message}")
        }
```

```

    }
```

/ **ServiceSync - Notification Se...** 2 Jira links

```
            if (successfullySentReservationNotifications.isNotEmpty()) {
                successfullySentReservationNotifications.forEach {
                    reservationDocumentMap[it]?.reference?.update(notificationSentFie
                }
            }
            else {
                throw Exception("Received 0 reservations to update!")
            }
        }
        catch (e : Exception) {
            println("Error updating notificationSentField. Error: ${e.message}")
        }
    }

    override fun getFcmToken(username: String): String {
         try {
            return firestore
                .collection(userCollectionName)
                .document(username)
                .get()
                .get()
                .getString(fcmTokenField) ?: ""
        }
        catch (e : Exception) {
            println("Error fetching fcm token of $username. Error: ${e.message}")
            return ""
        }

    }
}
```

This class has annotation @Repository in order to make it injectable into the business logic layer. It implements both the **IFcmTokenProvider and IReservationDataProvider** interfaces. All private properties are constants are are mostly used to know which collections and field names to fetch from firestore Inner logic of the methods:

- **getAllUnsentNotifications** - this method goes to the collection “reservations” and executes a complex query to fetch all documents that are greater than min threshold and equal or less than the max threshold. It converts the list to a list of reservations and keeps track of which document was translated into which reservation object so that later on they can be marked as sent. OnSuccess is only called if firestore returned at least 1 document
- **markNotificationAsSent -** this method updates the document references for each list member from the received list. It logs data if it was called with an empty list

- **getFcmToken** - this method goes to the collection “users”, finds the document (username is document id) and returns the value of field “FCMToken”. Logs data if there was an exception

/ **ServiceSync - Notification Se...** 2 Jira links

FirebaseMessagingManager

```
@Service
class FirebaseMessagingManager(private val firebaseMessaging: FirebaseMessaging) : IM

    override fun sendNotification(notificationData: List<NotificationData>, imageURL:
        val messages = mutableListOf<Message>()
        val notificationMapping = mutableMapOf<Message,NotificationData>()
        notificationData.forEach { notification ->
            if (notification.fcmToken.isNotEmpty() && notification.body.isNotEmpty()
                val data = hashMapOf(
                    "title" to notification.title,
                    "body" to notification.body
                )


                    val message = Message.builder()
                        .setToken(notification.fcmToken)
                        .setNotification(
                            Notification
                                .builder()
                                .setTitle(data["title"])
                                .setBody(data["body"])
                                .setImage(imageURL)
                                .build()
                        )
                        .build()

                    messages.add(message)
                    notificationMapping[message] = notification
            }

        }
        if (messages.isNotEmpty()) {
            val successfulNotifications = mutableListOf<NotificationData>()
            val batchResponse = firebaseMessaging.sendEach(messages)
            println("Number of successfully sent notifications: ${batchResponse.succe
            println("Number of failed notifications: ${batchResponse.failureCount}")
            val successfulResponses = batchResponse.responses.filter { it.exception =
            successfulResponses.forEachIndexed { index, sendResponse ->
                val message = messages[index]
                val notification = notificationMapping[message]
                notification?.let {
                    successfulNotifications.add(it)
                }
            }
```

```
            val failedResponses = batchResponse.responses.filter { it.exception != nu
            failedResponses.forEach {
```

/ **ServiceSync - Notification Se...** 2 Jira links

```
                onSuccessfullySentNotifications(successfulNotifications)
            }
        }
    }
}
```

- **sendNotification** - this method builds a message with title and body that was received form business logic layer, sets the image of the notification, adds a message the the list of messages and also to the map to keep track of transformation from NotificationData to Message. Then, sends every notification using built-int sendEach method, find all successfully sent messages and executes lambda function onSuccessfullySentNotifications. For those messages that could not have been sent, logs the cause of the exception (cause is more helpful than the actual exception message).

Business logic layer

ReservationService

```
@Service
class ReservationService(
    private val reservationDataProvider: IReservationDataProvider,
    private val fcmTokenProvider: IFcmTokenProvider,
    private val messagingProvider: IMessagingProvider,
) {
    private val imageName : String = "notification_image.png"

    @Scheduled(fixedRate = 3600000)
    fun checkForUpcomingReservations() {
        println("Checking for upcoming reservations...")
        var reservations = mutableListOf<Reservation>()
        val minThreshold = LocalDateTime.now().atZone(ZoneOffset.UTC).toInstant().toE
        val threshold = LocalDateTime.now().plus(Duration.ofHours(24)).atZone(ZoneOff
        reservationDataProvider.getAllUnsentNotifications(
            minThreshold =  minThreshold,
            maxThreshold = threshold,
            onSuccess = { unsentNotifications ->
                reservations = unsentNotifications.toMutableList()
            }
        )
        println("Found ${reservations.size} upcoming reservations that require sendir
        val notificationMapping = mutableMapOf<NotificationData,Reservation>()
        val allNotifications = mutableListOf<NotificationData>()
```

```
            reservations.forEach { reservation ->
                val fcm = fcmTokenProvider.getFcmToken(reservation.userId)
```

/ **ServiceSync - Notification Se...** 2 Jira links

```
                        fcmToken = fcm,
                        title = "Reservation reminder",
                        body =  "${reservation.serviceName} from ${reservation.compar
                    )
                    allNotifications.add(notificationData)
                    notificationMapping[notificationData] = reservation
                }
            }

            if (allNotifications.isNotEmpty()) {
                val pathToImage = Resources.getResource(imageName).path
                messagingProvider.sendNotification(allNotifications,pathToImage,
                    onSuccessfullySentNotifications = {
                        successfullNotifications ->
                        val sentReservations = mutableListOf<Reservation>()
                        successfullNotifications.forEach {
                            val reservation = notificationMapping[it]
                            reservation?.let {
                                sentReservations.add(reservation)
                            }
                        }
                        reservationDataProvider.markNotificationAsSent(sentReservatic

                    }

                )
            }


    }
}
```

This is the only business logic layer class with one method: **checkForUpcomingReservations**. As it is visible, this method is scheduled and will execute when the server is booted and **every 1 hour afterwards** (1 hour = 3 600 000 miliseconds in fixedRate). In order to learn advanced programming concepts, the server was developed using the D principle in SOLID principles - Dependency inversion. As you can see, this class requires 3 variables that implement 3 custom interfaces. Spring has built-in IoC container which can be used to leave Spring to handle injecting dependencies. The cool thing here is that no explicit configurations are needed. Why?. Because if only 1 class implements an interface, Spring can **automatically** find that class and will inject it because no other class exists. If there were more than 1 class, a file with the annotation @Configuration would be neede to tell Spring which one to inject. This is all possible with Spring annotations.

The method **checkForUpcomingReservations** sets the minimal threshold (current time of the server) and maximum threshold (current time + 24hrs) and calls the data access layer to execute

/ **ServiceSync - Notification Se...** 2 Jira links

body of the notification, gets the path to the image located in **main/resources** folder and passes the list to the data access layer. After receiving only the notificitaion data object that were successfully sent in the lambda function, it calls **markNotificationAsSent** data access layer method and awaits the method to finish.

SpringApplication component scan

```
@SpringBootApplication
@ComponentScan(basePackages = ["hr.foi.servicesync"])
class ControllersApplication

fun main(args: Array<String>) {
    runApplication<ControllersApplication>(*args)
}
```

Since the app is located in a different package than the ReservationService and other classes with annotations, there was a need to explicitly state where to scan for these components.

Diagrams

Package diagram

/ **ServiceSync - Notification Se...** 2 Jira links

Class Diagram

ServiceSync - Home
ServiceSync

Modular feature - Maps
ServiceSync

/ **ServiceSync - Notification Se...** 2 Jira links

Future appointments calendar
ServiceSync
More like this

Registration/Login - Product requirements
ServiceSync
More like this

Be the first to add a reaction